

Assignment ::- 7

Name	Abhineet Tiwari
Roll No:	20965
Subject	DBMS

Q1)

Create a MySQL database named CollegeDB and create two tables: Course and StudentResult.

StudentResult Table

Column Name	Data Type	Constraint
StudentID	INT	PRIMARY KEY
StudentName	VARCHAR(50)	NOT NULL
CourseID	INT	FOREIGN KEY
City	VARCHAR(30)	
Marks	INT	

Insert at least 4 records into Course and 10 records into StudentResult.

```
CREATE DATABASE CollegeDB;
```

```
USE CollegeDB;
```

```
CREATE TABLE Course (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50) NOT NULL,  
    Fees DECIMAL(10,2) NOT NULL  
);
```

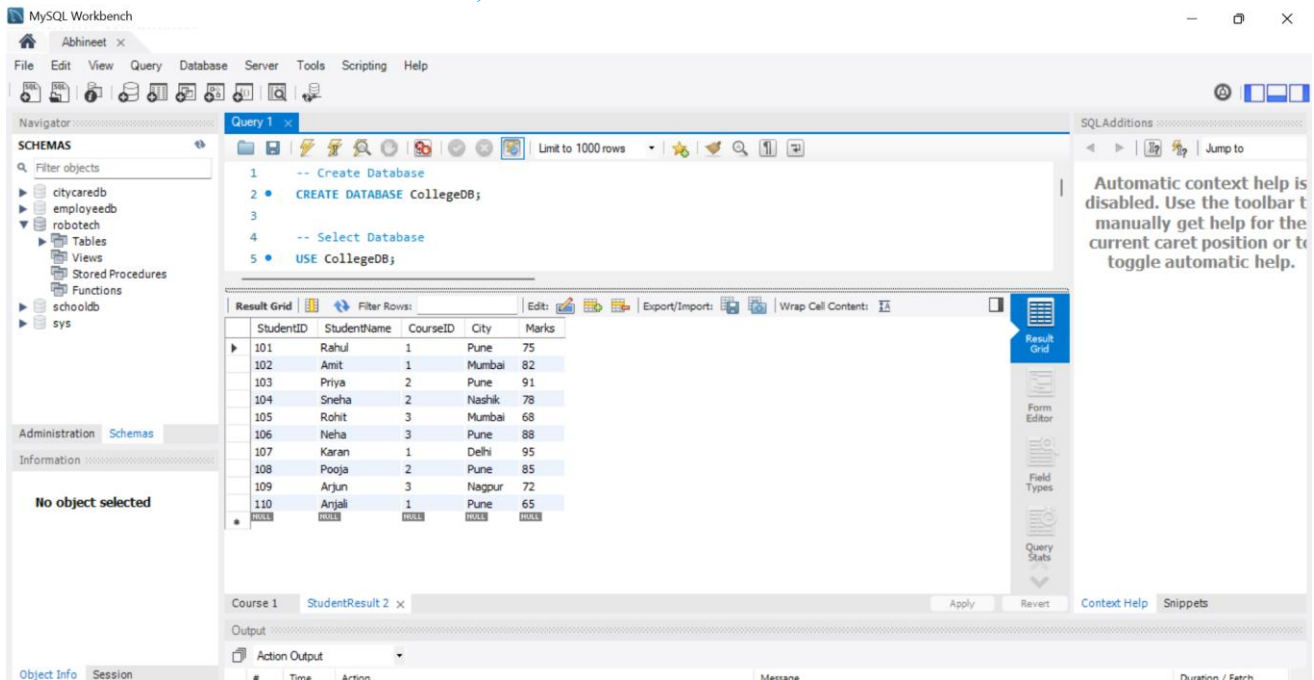
```
CREATE TABLE StudentResult (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50) NOT NULL,  
    CourseID INT,  
    City VARCHAR(30),  
    Marks INT,  
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```

```
INSERT INTO Course (CourseID, CourseName, Fees)  
VALUES  
(1, 'BCA', 50000.00),  
(2, 'B.SC. DATA SCIENCE', 60000.00),  
(3, 'BBA', 45000.00),  
(4, 'B.COM', 40000.00);  
INSERT INTO StudentResult  
(StudentID, StudentName, CourseID, City, Marks)  
VALUES  
(101, 'Rahul', 1, 'Pune', 75),  
(102, 'Amit', 1, 'Mumbai', 82),  
(103, 'Priya', 2, 'Pune', 91),
```

```
(104, 'Sneha', 2, 'Nashik', 78),
(105, 'Rohit', 3, 'Mumbai', 68),
(106, 'Neha', 3, 'Pune', 88),
(107, 'Karan', 1, 'Delhi', 95),
(108, 'Pooja', 2, 'Pune', 85),
(109, 'Arjun', 3, 'Nagpur', 72),
(110, 'Anjali', 1, 'Pune', 65);
```

```
SELECT * FROM Course;
```

```
SELECT * FROM StudentResult;
```



Q2) Write a MySQL query to display students whose marks are greater than the average marks of all students.

The output should contain:

Column
StudentName
Marks

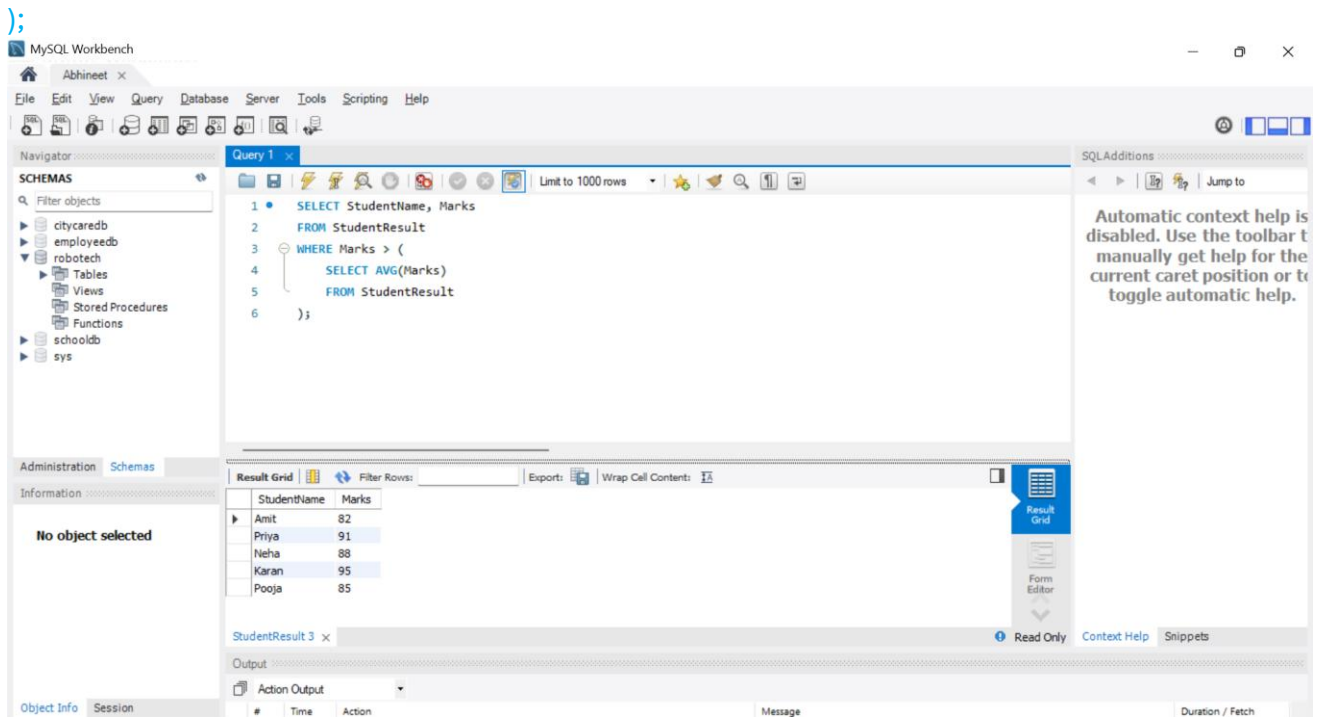
```
SELECT StudentName, Marks
```

```
FROM StudentResult
```

```
WHERE Marks > (
```

```
SELECT AVG(Marks)
```

```
FROM StudentResult
```



Q3) Write a MySQL query to display all students enrolled in the BCA course using a subquery.

The output should contain:

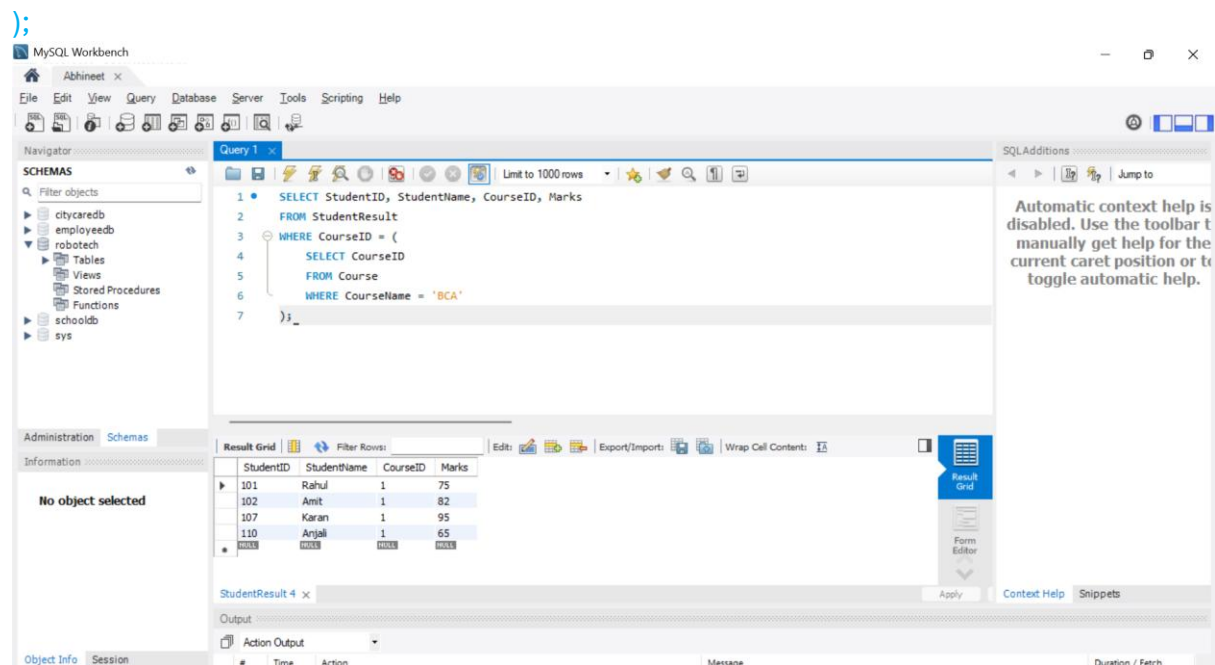
Column
StudentID
StudentName

Column
CourseID
Marks

```

SELECT StudentID, StudentName, CourseID, Marks
FROM StudentResult
WHERE CourseID = (
    SELECT CourseID
    FROM Course
    WHERE CourseName = 'BCA'
);

```



Q4) Write a MySQL query to display each student with the overall average marks using a subquery in the SELECT clause.

The output should contain:

Column
StudentName
Marks
AverageMarks

```

SELECT
    StudentName,
    Marks,
    (SELECT AVG(Marks) FROM StudentResult) AS AverageMarks

```

FROM StudentResult;

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL query:

```
1 SELECT
2   StudentName,
3   Marks,
4   (SELECT AVG(Marks) FROM StudentResult) AS AverageMarks
5 FROM StudentResult;
```

The 'Result Grid' shows the output of the query:

StudentName	Marks	AverageMarks
Rahul	75	79.9000
Amit	82	79.9000
Priya	91	79.9000
Sneha	78	79.9000
Rohit	68	79.9000
Neha	88	79.9000
Karan	95	79.9000
Pooja	85	79.9000

The 'Output' tab is also visible, showing 'Action Output'.

Q5) Write a MySQL query using a correlated subquery to display students who scored more than their own course average.

The output should contain:

Column
StudentID
StudentName
CourseID
Marks

SELECT

StudentID,

StudentName,

CourseID,

Marks

FROM StudentResult s

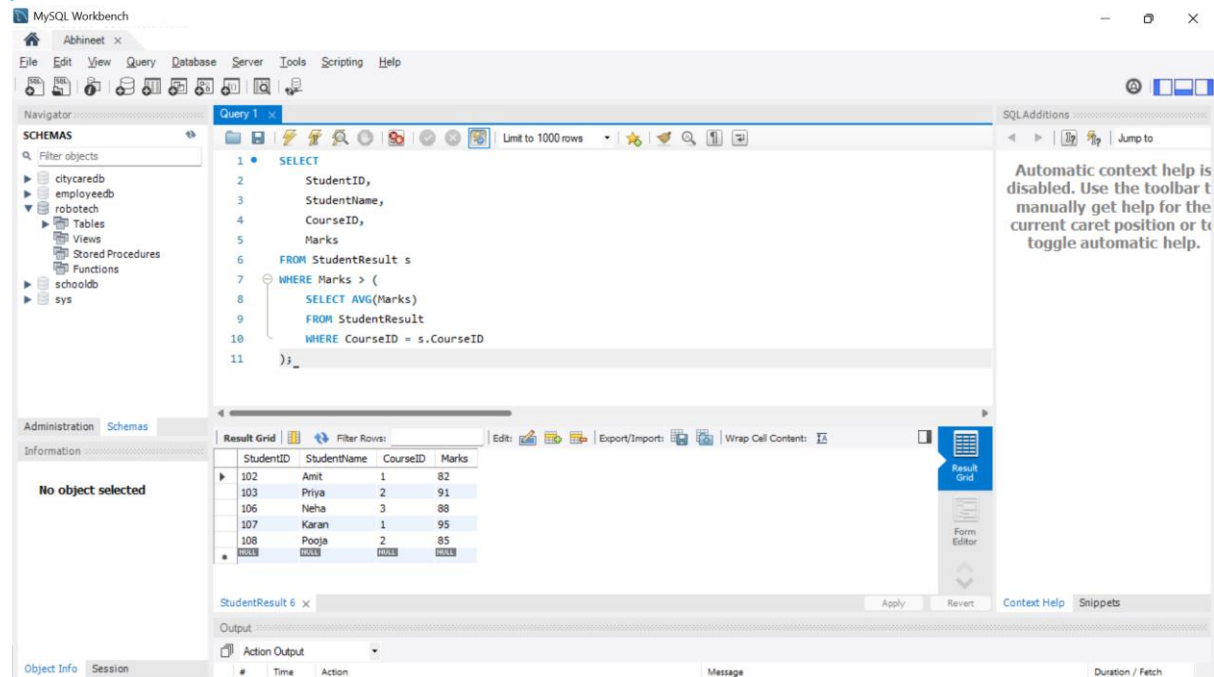
WHERE Marks > (

SELECT AVG(Marks)

FROM StudentResult

WHERE CourseID = s.CourseID

);



Q6) Write MySQL queries to display courses having students and courses having no students.

Use:

Operator
EXISTS
NOT EXISTS

SELECT CourseID, CourseName

FROM Course c

WHERE EXISTS (

SELECT 1

FROM StudentResult s

WHERE s.CourseID = c.CourseID

);

The screenshot displays the MySQL Workbench interface. The 'Query Editor' window shows the following SQL query:

```
1 SELECT CourseID, CourseName
2 FROM Course c
3 WHERE EXISTS (
4   SELECT 1
5   FROM StudentResult s
6   WHERE s.CourseID = c.CourseID
7 );
```

The 'Result Grid' window shows the results of the query:

CourseID	CourseName
1	BCA
2	B.SC. DATA SCIENCE
3	BBA
4	

The 'Navigator' panel on the left shows the database schema, including tables, views, stored procedures, functions, and schemas. The 'Output' panel at the bottom shows the 'Action Output'.

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.